DropLink Project Guide

1) Name check and recommendation

Quick web checks show that:

- **ShareDrop** is already an existing open-source web app for file sharing.
- **Snapdrop** is also already taken, and the GitHub README says it is now LimeWire.
- Several obvious transfer-style names are also already in use online, including **DropLink**, **PeerDrop**, **ZipBeam**, and **AirBridge**.

Safer original name candidates

These are better as fresh brand ideas, but they still need a full trademark/domain check before you commit:

- **SignalCrate**
- **MosaicBeam**
- **RelayMint**
- **QuiltSend**
- **TetherFerry**

My strongest pick for your idea: **SignalCrate**. It feels technical, readable, and not obviously borrowed from an existing file-sharing brand.

---

2) Project summary

Project goal: build a browser-based PWA that lets a user send or receive files between phone and PC using QR code or link, with temporary storage, automatic cleanup, optional ZIP for bulk uploads, and a clean privacy-first UX.

Core promise: - No USB cable - No WhatsApp Web login - No app install for the receiver - No account for basic use - QR or link based transfer - Works on any modern browser

---

3) Tech stack

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS
- shadcn/ui
- TanStack Query
- Axios
- React Hook Form
- Zod
- next-pwa
- qrcode.react
- JSZip

Backend

- Node.js
- Express
- TypeScript

- MongoDB Atlas
- Mongoose
- Socket.IO
- Multer
- Archiver
- Zod
- Helmet
- CORS
- express-rate-limit
- Compression
- UUID
- node-cron

Tooling

- Git
- GitHub
- VS Code
- Postman
- PowerShell 7
- MongoDB Atlas
- npm or pnpm

---

4) Repository structure

```
droplink/
|-- frontend/
|   |-- app/
|   |-- components/
|   |-- features/
|   |-- lib/
|   |-- public/
|   |-- styles/
|   |-- types/
|   |-- .env.example
|   |-- next.config.ts
|   |-- package.json
|   `-- tsconfig.json
|-- backend/
|   |-- src/
|   |   |-- config/
|   |   |-- middleware/
|   |   |-- models/
|   |   |-- modules/
|   |   |   |-- upload/
|   |   |   |-- transfer/
|   |   |   |-- download/
|   |   |   `-- cleanup/
|   |   |-- routes/
|   |   |-- services/
|   |   |-- utils/
|   |   `-- server.ts
|   |-- uploads/
```

```
|   |-- .env.example
|   |-- package.json
|   `-- tsconfig.json
|-- docs/
`-- README.md
```

This matches the split backend/frontend structure and separate env-example files from your project blueprint.

---

5) What to install

Required

- Node.js LTS
- Git
- VS Code
- MongoDB Atlas account
- PowerShell 7 or Windows PowerShell
- Postman

Frontend packages

```
npm i axios zod @tanstack/react-query react-hook-form @hookform/resolvers qrcode.react jszip next-pwa
```

Backend packages

```
npm i express mongoose cors helmet express-rate-limit zod dotenv multer socket.io compression uuid archi
npm i bcryptjs jsonwebtoken
npm i -D typescript tsx nodemon @types/node @types/express @types/cors @types/multer @types/uuid @types/
```

---

6) PowerShell setup commands

Create the root folder

```
mkdir DropLink
cd DropLink
mkdir frontend, backend, docs
```

Create the main folder tree

```
mkdir frontend\app, frontend\components, frontend\features, frontend\lib, frontend\public, frontend\styl
mkdir backend\src, backend\src\config, backend\src\middleware, backend\src\models, backend\src\modules,
```

Create empty starter files

```
New-Item frontend\.env.example -ItemType File -Force
New-Item frontend\package.json -ItemType File -Force
New-Item frontend\tsconfig.json -ItemType File -Force
New-Item frontend\next.config.ts -ItemType File -Force

New-Item backend\.env.example -ItemType File -Force
New-Item backend\package.json -ItemType File -Force
New-Item backend\tsconfig.json -ItemType File -Force
New-Item backend\src\server.ts -ItemType File -Force
New-Item backend\src\config\env.ts -ItemType File -Force
```

```
New-Item backend\src\config\db.ts -ItemType File -Force

New-Item README.md -ItemType File -Force
New-Item docs\PRD.md -ItemType File -Force
```

Initialize frontend

```
cd frontend
npx create-next-app@latest . --typescript --tailwind --eslint --app --src-dir --import-alias "@/*"
npm i axios zod @tanstack/react-query react-hook-form @hookform/resolvers qrcode.react jszip next-pwa
cd ..
```

Initialize backend

```
cd backend
npm init -y
npm i express mongoose cors helmet express-rate-limit zod dotenv multer socket.io compression uuid archi
npm i bcryptjs jsonwebtoken
npm i -D typescript tsx nodemon @types/node @types/express @types/cors @types/multer @types/uuid @types/
npx tsc --init
cd ..
```

Run both apps

```
cd frontend
npm run dev
```

Open a second terminal:

```
cd backend
npm run dev
```

---

7) Environment files

backend/.env.example

```
NODE_ENV=development
PORT=5000
MONGODB_URI=mongodb://localhost:27017/droplink

CLIENT_URL=http://localhost:3000
UPLOAD_DIR=uploads

MAX_FILE_SIZE=2147483648
MAX_FILES=100
TRANSFER_EXPIRY_MINUTES=10
MAX_DOWNLOADS=1

JWT_SECRET=replace_with_a_long_random_secret
LOG_LEVEL=info
```

frontend/.env.example

```
NEXT_PUBLIC_API_URL=http://localhost:5000
NEXT_PUBLIC_APP_NAME=DropLink
```

```
NEXT_PUBLIC_MAX_UPLOAD=2147483648
```

---

8) MVP execution flow

Sender flow

1. Open `/send`
2. Select one file or multiple files
3. If multiple files, zip automatically
4. Upload to backend
5. Backend creates a transfer session
6. Frontend shows QR code and share link
7. Receiver opens link or scans QR
8. Receiver downloads file
9. Backend deletes the files after download or expiry

Receiver flow

1. Open `/receive`
2. Scan QR or paste link
3. Check transfer status
4. Download file
5. Session expires automatically

---

9) Functional requirements

Send

- Single file upload
- Multi-file upload
- Auto ZIP for bulk files
- Progress bar
- Cancel upload
- QR generation
- Shareable link generation

Receive

- QR scan support
- Link open support
- Download page
- Progress indicator
- Transfer expiry warning

Backend

- Temporary file storage
- Transfer session creation
- Cleanup job
- Download counter
- Expiry handling
- Validation and rate limiting

PWA

- Installable
- App icon and manifest
- Offline shell for home page
- Mobile-friendly send/receive screens

---

10) Non-functional requirements

- Fast first load
- Responsive UI
- Temporary secure storage
- Automatic cleanup
- Basic abuse protection
- Simple UX for college use
- Works on mobile and desktop browsers

---

11) PRD — Product Requirements Document

Product name

DropLink

Problem statement

People often need to move files between phone and PC quickly, but they do not always have a data cable, and they do not want to log in to chat apps or cloud drives on shared machines.

Target users

- College students
- Office workers
- Teachers
- Lab users
- Anyone transferring files between personal devices and shared computers

Primary use cases

- Phone to college PC
- PC to phone
- One-off document transfer
- Bulk image transfer
- Small project file handoff

Goals

- Make file transfer faster than email and less risky than shared cloud links
- Remove account friction
- Make the receiver flow dead simple
- Auto-delete stale transfers

Non-goals for MVP

- Full peer-to-peer WebRTC transfer
- End-to-end encryption in v1
- User accounts
- Chat
- File sync
- Folder sync
- Native mobile apps

Core user stories

- As a sender, I want to upload a file and get a QR code so another device can download it.
- As a receiver, I want to open a link or scan a QR code so I can get the file immediately.
- As a sender, I want multiple files zipped automatically so I can send one clean package.
- As an admin, I want expired files deleted automatically so storage does not keep growing.

Acceptance criteria

- Upload one or more files successfully
- Generate a share link and QR
- Receiver can download from another device
- Files are deleted after download or after expiry
- PWA installs on mobile
- UI works on phone and desktop widths

Risks

- Large file uploads can fail on unstable networks
- Temporary disk storage can fill up if cleanup fails
- Public QR links can be forwarded unless they expire quickly
- College firewall policies can block some requests

Metrics

- Upload success rate
- Download success rate
- Average time to share
- Percentage of expired transfers cleaned up
- Mobile PWA install count

Roadmap

Phase 1: upload, QR, link, download, cleanup
Phase 2: password protection, transfer history
Phase 3: WebRTC, end-to-end encryption, resumable transfers

---

12) Recommended first build order

1. Create repo and folder structure
2. Set up Next.js frontend
3. Set up Express backend
4. Add env validation
5. Build upload endpoint
6. Save files temporarily

7. Generate transfer session
8. Show QR and link
9. Build download route
10. Add cleanup job
11. Add PWA manifest and installability
12. Polish UI and test on phone + PC